

COMPUTER ARCHITECTURE AND ORGANIZATION

LAB ASSIGNMENT - 09 (MIPS PROCESSOR IMPLEMENTATION WITH MEMORY)

1. UNDERSTANDING OF THE TASK :

The task is to extend the single-cycle MIPS processor from the previous assignment into a complete processor in Logisim that supports instruction fetching and memory operations. The design now includes a Program Counter (PC), Instruction Memory (IM), and Data Memory (DM) to execute multiple instructions sequentially. It supports R-type (add, sub, and, or, slt), I-type (addi, andi, ori), branch (beq), and memory (lw, sw) instructions. The system was modularly built using subcircuits such as the ALU, ALU Control Unit, Control Unit, Register File, multiplexers, and supporting components like the Sign Extend, Left Shift, Instruction Memory, Data Memory, and MemtoReg MUX, all of which were integrated into the main processor circuit.

2. DESCRIPTION OF SUBMITTED .CIRC FILES :

- **B23CS1052_main.circ** : This is the top-level circuit integrating all other modules of the processor. It connects the Control Unit, ALU, ALU Control Unit, Register File, multiplexers, and supporting components. The circuit includes a Program Counter (PC), Instruction Memory (IM), and Data Memory (DM) to enable full instruction fetching and execution. Instructions are automatically fetched from the Instruction Memory using the PC, and data is read from or written to the Data Memory as required.
- **B23CS1052_ControlUnit.circ**: Generates all necessary control signals based on the opcode of the instruction. It differentiates between R-type, I-type (immediate), memory access (lw, sw), and branch (beq) instructions. The unit outputs control signals such as RegDst, ALUSrc, RegWrite, Branch, MemRead, MemWrite, MemtoReg, and ALUOp to coordinate data flow and enable proper operation of the processor's datapath.
- **B23CS1052_RegisterFile.circ** : This circuit implements the 32×32 -bit register file used in the MIPS processor. It supports two read operations and one write operation per clock cycle. The inputs are read_reg1 and read_reg2 (5-bit register addresses for reading), write_reg (5-bit destination register address), write_data (32-bit data to be written), regwrite (control signal to enable writing), and the clock input which synchronizes write operations. The outputs are read_data1 and read_data2, providing 32-bit data read from the registers specified by read_reg1 and read_reg2. This module supplies operands to the ALU and stores the results of executed instructions.
- **B23CS1052_RegDst_Mux.circ**: A multiplexer that selects the destination register for writing results back to the Register File. For R-type instructions, it selects the rd field, while for I-type and memory (lw) instructions, it selects the rt field, based on the RegDst control signal generated by the Control Unit.

- **B23CS1052_ALU.circ:** This circuit implements the Arithmetic Logic Unit (ALU) of the MIPS processor. It performs arithmetic and logical operations such as ADD, SUB, SLT, AND, and OR, as well as immediate operations like ADDI, ANDI, ORI. The ALU takes two 32-bit input operands and an ALUControl signal, based on which it selects the appropriate operation to perform. It also calculates the effective memory address for load (lw) and store (sw) instructions. The ALU generates a Zero flag output, which becomes 1 when the ALU result is zero, and is primarily used by the beq instruction to determine branch conditions.
- **B23CS1052_ALUSrc_Mux.circ:** Selects the second input to the ALU either the register operand (for R-type instructions) or the sign-extended immediate value (for I-type and memory access instructions such as lw and sw), based on the ALUSrc control signal. This allows the ALU to perform both register-based and immediate-based operations within the processor's datapath.
- **B23CS1052_SignExtend.circ:** Extends the 16-bit immediate value from the instruction to a 32-bit signed number. This is essential for immediate arithmetic instructions (addi), memory access instructions (lw, sw), and for calculating branch target offsets used in beq instructions.
- **B23CS1052_ALUControl.circ:** This circuit generates the specific control signals required by the ALU. It takes three main inputs — the ALUOp signal from the Control Unit, the funct field from the instruction, and, in some cases, the opcode. For R-type instructions, both the ALUOp and funct field determine operations such as ADD, SUB, AND, OR, or SLT. For I-type and memory instructions (addi, lw, sw), the operation is determined solely by the ALUOp signal. However, for logical immediate instructions such as andi and ori, both share the same ALUOp value (11), but since their required ALU control signals differ, the opcode is also used to distinguish between them. This ensures the correct ALU operation is selected for arithmetic, logical, memory, and branch instructions.
- **B23CS1052_BranchControl.circ :** This circuit determines whether a branch should be taken for the beq instruction. It takes the Branch control signal from the Control Unit and the Zero flag from the ALU as inputs. These two signals are combined using an AND gate to produce a single output signal named PCSrc, which indicates whether the branch condition is satisfied. When both the Branch signal and the Zero flag are high, the AND gate output (PCSrc) becomes 1, signifying that the branch should be taken. Otherwise, PCSrc remains 0, and normal sequential execution continues.
- **B23CS1052_Branch_Address_Calculation.circ:** This circuit calculates the target address for the beq instruction by adding the left-shifted, sign-extended branch offset to PC + 4. The resulting branch target address is sent to the Next PC MUX, which selects it as the next value for the Program Counter when the PCSrc signal is 1, indicating that the branch condition is true.
- **B23CS1052_LeftShiftBy2.circ :** Shifts the sign-extended immediate value left by two bits. This aligns the branch offset correctly since MIPS branch addresses are word-aligned.
- **B23CS1052_InstructionMemory.circ:** This circuit contains the Instruction Memory (IM), implemented using a RAM component with a 10-bit address width. The address input is connected to PC[13:2] to ensure word alignment,

and the memory outputs the 32-bit instruction corresponding to the current PC value.

- **B23CS1052_DataMemory.circ** : This circuit contains the Data Memory (DM), implemented using a RAM component to store and retrieve data during program execution. The address input is provided by the ALU result, while the data input for write operations comes from the Register File. The memory outputs data for load (lw) instructions and writes data for store (sw) instructions, controlled by the MemRead and MemWrite signals.
- **B23CS1052_MemtoReg_Mux.circ**: This circuit implements the MemtoReg multiplexer, which selects the data to be written back to the Register File. It chooses between the ALU result (for R-type and immediate instructions) and the Data Memory output (for load word instructions), based on the MemtoReg control signal generated by the Control Unit.
- **Program Counter** : The Program Counter (PC) is implemented in the top-level circuit, where it holds the address of the current instruction and updates every clock cycle based on the PC update logic and branch control signals.

3. NON-TRIVIAL LOGIC OF NEW DESIGN :

- a. **PROGRAM COUNTER** : The Program Counter (PC) is a 32-bit register that holds the address of the instruction to be fetched from instruction memory. It updates every clock cycle based on the control signal PCSrc, which determines whether the processor continues sequentially or branches to a target address.

PCSrc Control Signal	PC input
PCSrc = 0	PC+4
PCSrc = 1	$(PC + 4) + (\text{SignExt}(\text{imm}) \ll 2)$

Output of PC : The 32-bit current instruction address, used as input to the Instruction Memory. Since instruction memory is word-aligned, PC[11:2] is connected as the address input. The address width of the Instruction Memory is 10 bits.

- b. **INSTRUCTION MEMORY** : The Instruction Memory (IM) stores the machine code of the program to be executed by the processor. It is implemented using a RAM component in Logisim and contains the sequence of 32-bit instructions that the processor fetches based on the Program Counter (PC) value.

Instruction Memory Input : The 10-bit address input is derived from PC[11:2], as instruction memory is word-aligned (each instruction is 4 bytes).

Load = 1, Chip Select = 1, and Clear = 0 are used to enable memory read operations. Since instructions are only fetched and not modified, the memory operates in read-only mode during execution.

Instruction Memory Output : The 32-bit instruction read from the specified memory address. This output is divided into appropriate fields opcode, rs, rt, rd, funct, and immediate for decoding and control signal generation.

c. **CONTROL UNIT :**

Control unit input : opcode of Instruction.

Instruction	Reg Dst	ALU Src	Memto Reg	Reg write	Mem Read	Mem Write	Branch	ALU op
add	1	0	0	1	0	0	0	10
sub	1	0	0	1	0	0	0	10
slt	1	0	0	1	0	0	0	10
and	1	0	0	1	0	0	0	10
or	1	0	0	1	0	0	0	10
addi	0	1	0	1	0	0	0	00
andi	0	1	0	1	0	0	0	11
ori	0	1	0	1	0	0	0	11
beq	X	0	X	0	0	0	1	01
lw	0	1	1	1	1	0	0	00
sw	X	1	X	0	0	1	0	00

Opcode Decoding : For each instruction, a separate 6-bit AND gate is implemented, where opcode bits are connected such that 0's are inverted and 1's are direct. This ensures that only the AND gate of the corresponding instruction activates when its opcode is given as input. The outputs of these gates are then combined logically to generate all required control signals for the datapath. Each control signal is generated using OR gates that combine the outputs of specific instruction-detecting AND gates.

1.RegDst : The RegDst signal is asserted only for R-type instructions (opcode 000000).

Hence, its OR gate input is connected solely to the output of the R-type instruction AND gate.

2.ALUSrc : The ALUSrc signal is asserted for instructions that use an immediate or memory address as the second ALU operand specifically addi, andi, ori, lw, and sw.

Hence, the OR gate for ALUSrc takes inputs from the AND gates corresponding to these five instructions.

ALUSrc = addi v andi v ori v lw v sw (v = OR operation)

3.MemtoReg : Only the lw (load word) instruction asserts this signal to 1, so data from Data Memory is written back to the Register File instead of the ALU result.

MemtoReg = lw

4.RegWrite : This signal is asserted for all instructions that write results back to the Register File — including R-type, immediate, and load instructions.

RegWrite = R-type v addi v andi v ori v lw

5.MemRead : The MemRead signal is asserted only for the load word (lw) instruction, which requires reading data from Data Memory.

MemRead = lw

6.MemWrite : The MemWrite signal is asserted only for the store word (sw) instruction, which writes data from the Register File into Data Memory.

MemWrite = sw

7.Branch : The Branch signal is asserted only for the branch-equal (beq) instruction.

Branch = beq

8.ALUOp : The ALUOp signal is 2 bits wide (ALUOp[1:0]) and determines the type of operation the ALU should perform.

It is generated using OR gates based on the instruction type:

- **ALUOp[0]** : Activated when R-type, andi, or ori instruction detectors are High
ALUOp[0] = R-type v andi v ori
- **ALUOp[1]** : Activated when andi, ori, or beq instruction detectors are High.
ALUOp[1] = andi v ori v beq
- All other instructions (lw, sw, addi) produce **ALUOp = 00**

d. DATA MEMORY :

INPUT :

The Data Memory uses a 10-bit address input from ALU[11:2], as addresses are word-aligned.

The Write Data (32 bits) input is connected to the ReadData2 output of the Register File (the value to be stored during sw).

- The Load pin of the RAM is connected to MemRead, asserted to perform a read (output data placed on ReadData).
- The Store pin of the RAM is connected to MemWrite, asserted to perform a write (write data stored into memory).
- The Sel / ChipSelect pin of the RAM is connected to the result of (MemRead OR MemWrite) — an OR gate combines the memory access signals to enable the RAM chip whenever it is read from or written to.

OUTPUT:

The ReadData output provides the data read from memory, which is driven back to the Register File when MemRead is asserted and selected by the MemtoReg MUX

e. MEMTOREG MUX :

The MemtoReg MUX selects the data to be written back into the Register File.

It has two 32-bit inputs:

- Input 0: ALU Result (for R-type and I-type arithmetic/logical instructions).
- Input 1: Read Data from Data Memory (for lw instruction).

MemtoReg	Data written back to Register File comes from
0	ALU result
1	Read Data from Data Memory

4. MODIFICATIONS MADE TO ASSIGNMENT 08 COMPONENTS :

MODIFICATIONS MADE TO CONTROL UNIT :

- In the Control Unit, new signals MemRead, MemWrite, and MemtoReg were added to support memory operations, and the ALUSrc logic was updated to include these instructions.
- The ALUOp circuit also did not require any changes.
- The logic remains the same: ALUOp[0] is asserted for R-type, andi, and ori instructions, while ALUOp[1] is asserted only for andi, ori, and beq instructions.
- For lw and sw, both bits remain 0, resulting in ALUOp = 00.

MIPS TEST PROGRAMS

.ASM CODE OF TEST PROGRAM 1:

```
.text
.globl main

main:

    # Store 10 at M[100] (0x64)
    addi $t0, $zero, 10
    sw $t0, 100($zero)

    # Store 20 at M[104] (0x68)
    addi $t0, $zero, 20
    sw $t0, 104($zero)

    # Store 30 at M[108] (0x6C)
    addi $t0, $zero, 30
    sw $t0, 108($zero)

    # Load A (10) into $s0
    lw $s0, 100($zero)

    # Load B (20) into $s1
    lw $s1, 104($zero)

    # Load C (30) into $s2
    lw $s2, 108($zero)

    # We must wait for the 'lw' instructions to finish

    sll $zero, $zero, 0    # nop
    sll $zero, $zero, 0    # nop
    sll $zero, $zero, 0    # nop

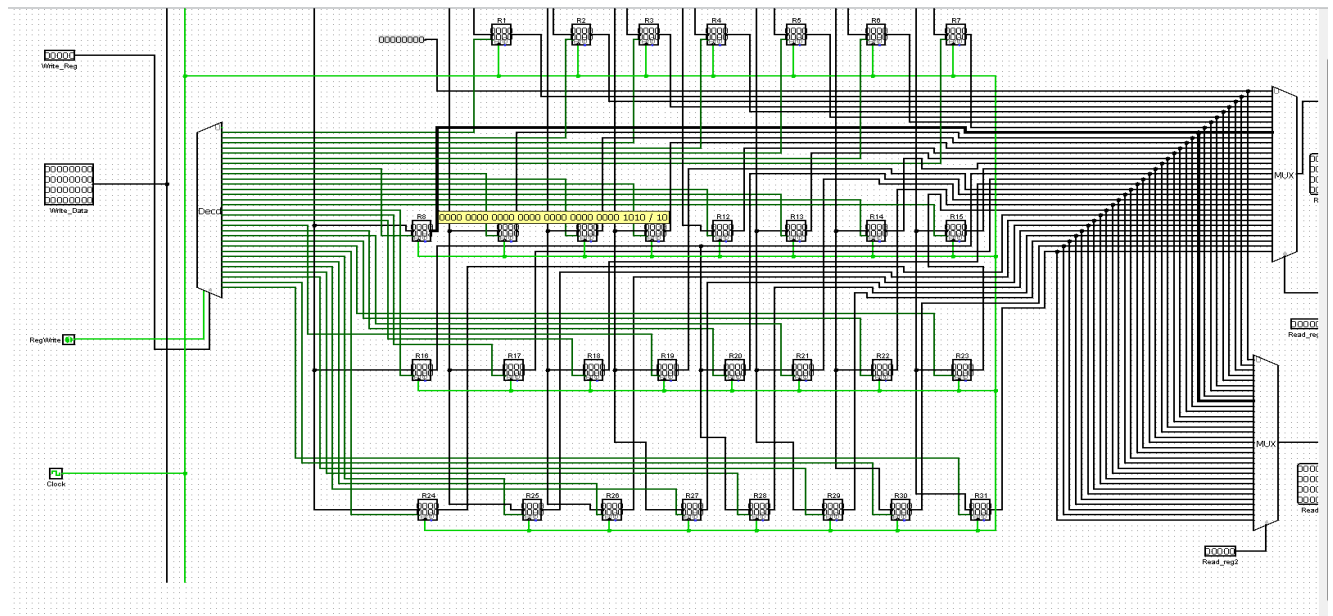
    # $t1 = A + B (10 + 20)
    add $t1, $s0, $s1

    # $s3 = $t1 + C (30 + 30)
    add $s3, $t1, $s2      # $s3 should now be 60

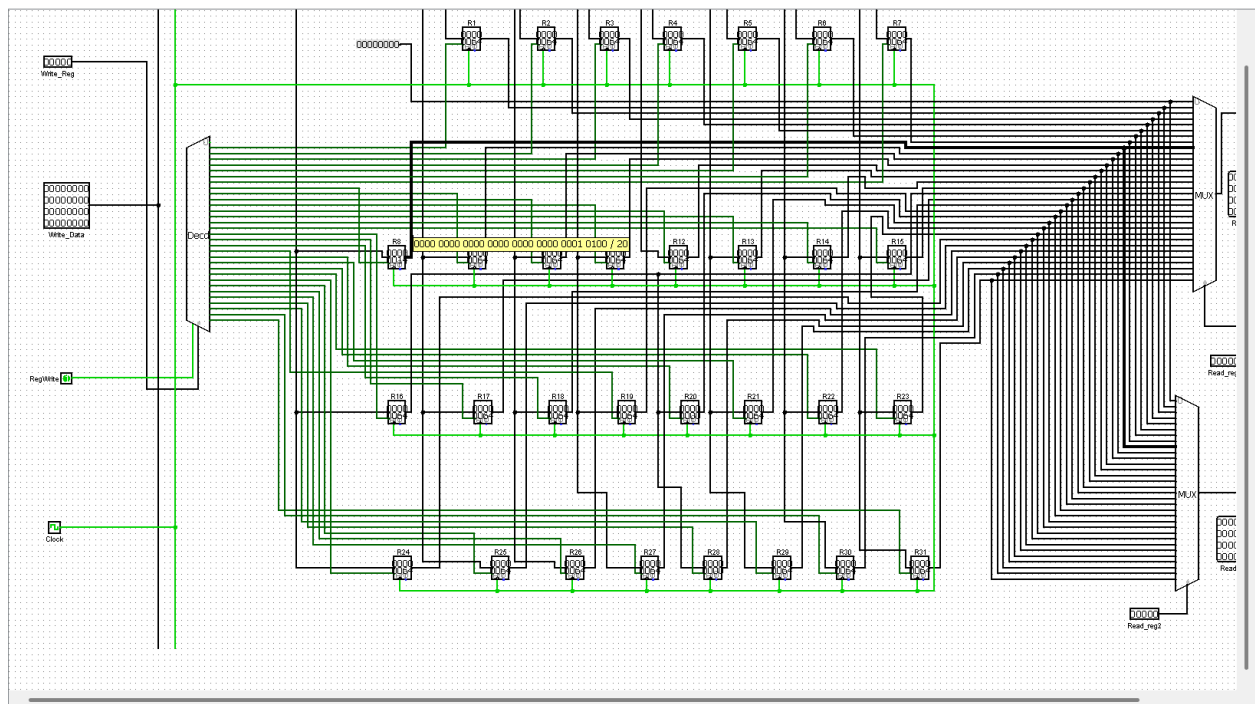
    # Store 60 (0x3c) at M[112] (0x70)
    sw $s3, 112($zero)

    # --- Infinite Halt ---
halt:
    beq $zero, $zero, halt
```

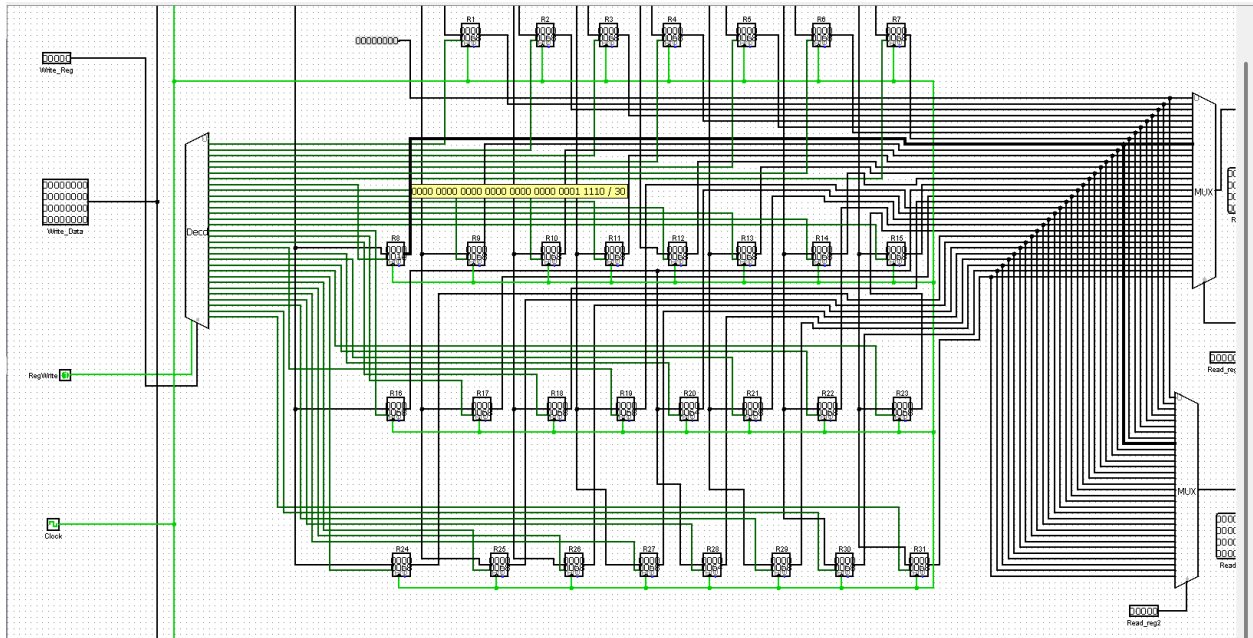
REGISTER FILE SIMULATION PICTURES :



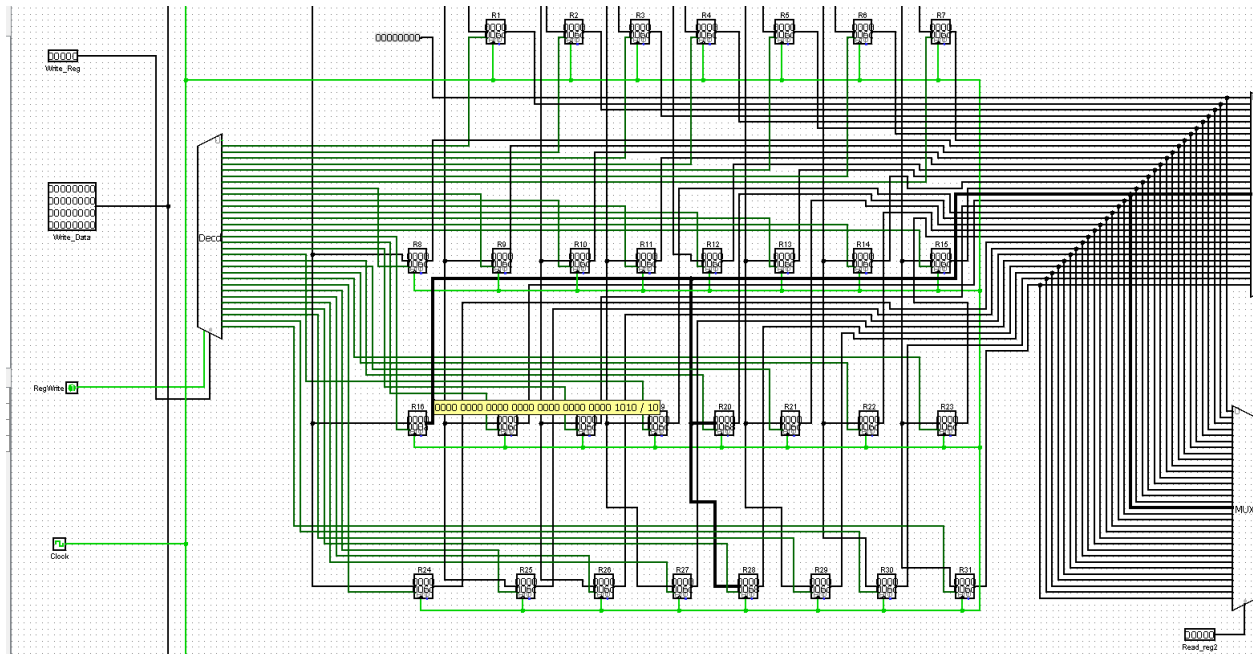
After executing `addi $t0, $zero, 10`
 \$t0 (R8) should contain value 10



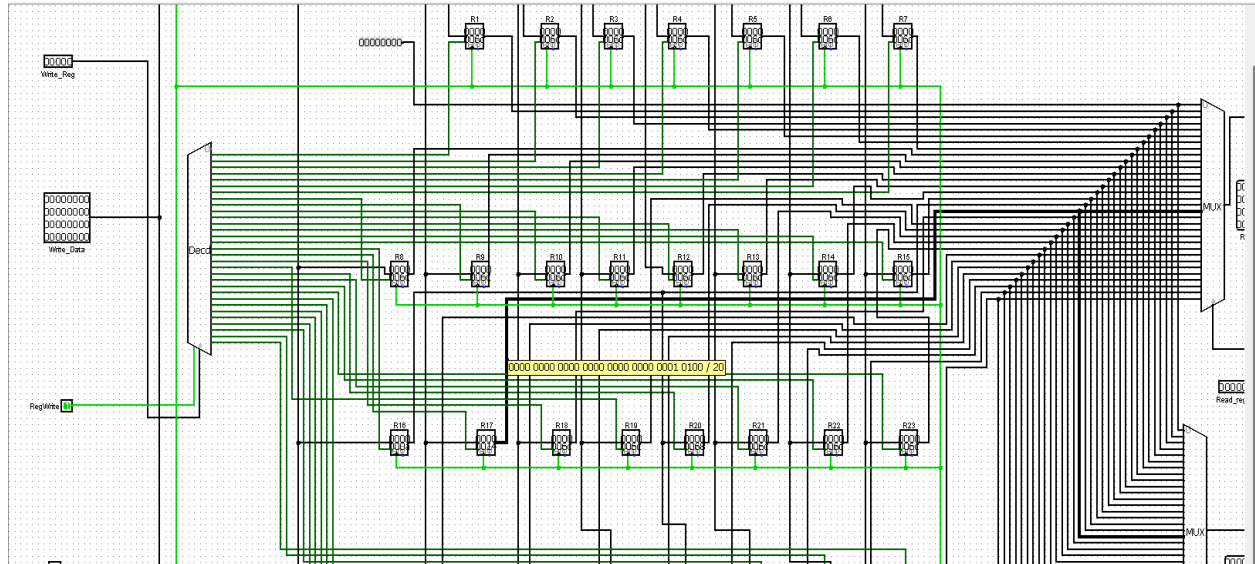
After executing `addi $t0, $zero, 20`
 \$t0(R8) should contain value 20



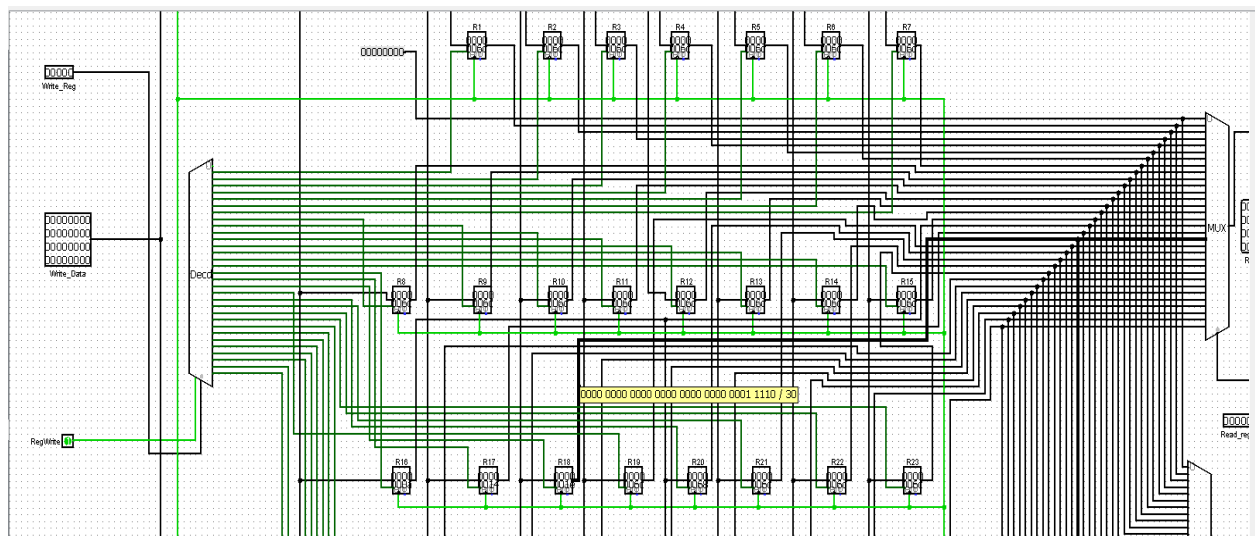
After executing `addi $t0, $zero, 30`
`$t0(R8)` should contain value 30



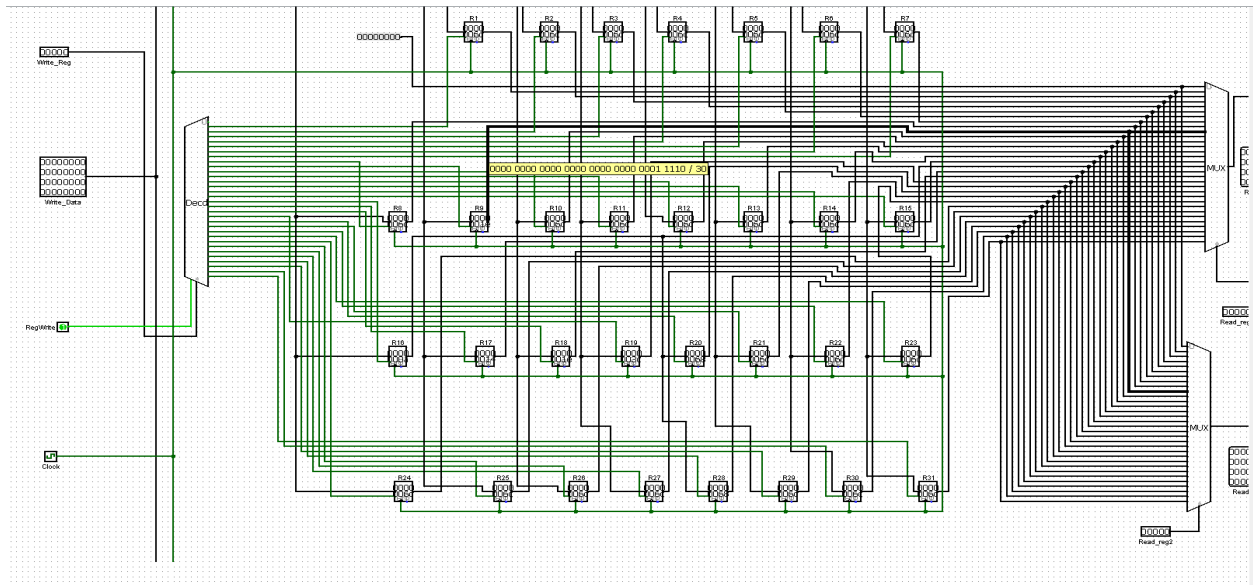
After executing `lw $s0, 100($zero)`
`$s0 (R16)` should contain 10



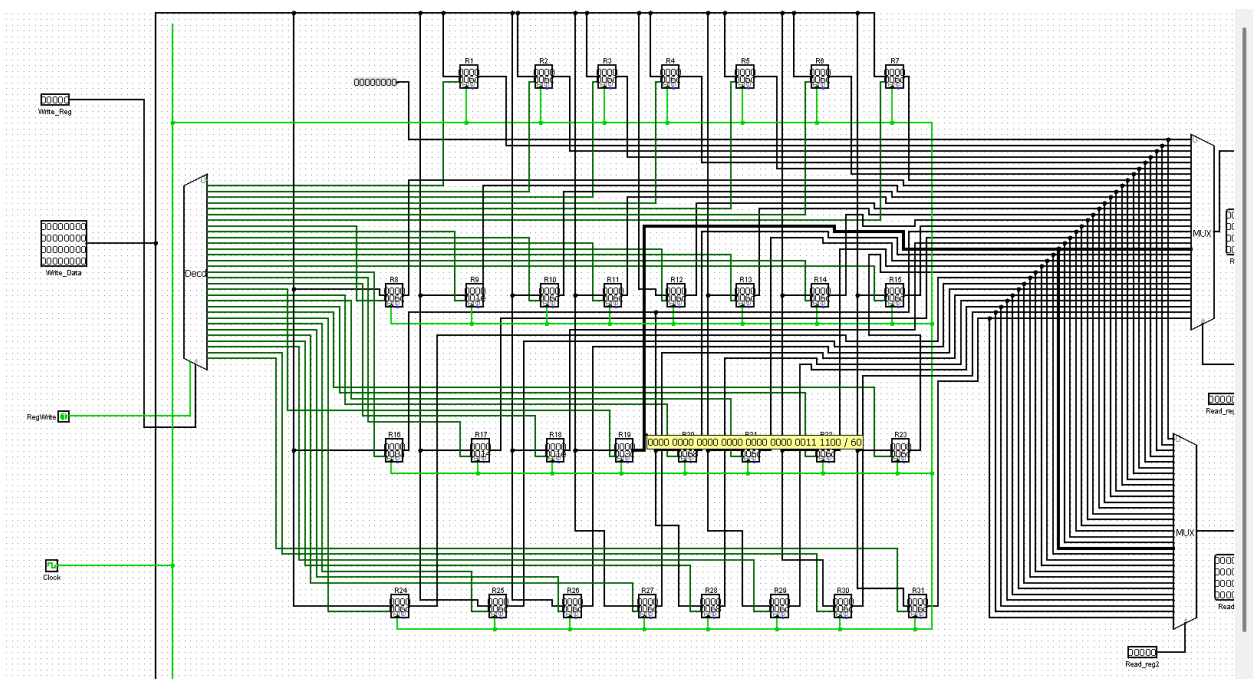
After Executing `lw $s1, 104($zero)`
`$s1(R17)` should contain 20



After Executing `lw $s2, 108($zero)`
`$s2(R18)` should contain 30



After Executing `add $t1, $s0, $s1`
`$t1` should be 30



After executing `add $s3, $t1, $s2`
`$s3(19)` should contain 60.

DATA MEMORY SIMULATION :

```

000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
008 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
010 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
018 00000000 0000000a 00000014 0000001e 0000003c 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
028 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
038 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
040 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
048 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
050 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
058 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
060 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
068 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
070 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
078 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
080 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
088 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

Byte address (Decimal)	Word Address (Decimal)	Word Address (Hex)	Stored Value (Hex)
100	25	0x19	0x0a
104	26	0x1A	0x14
108	27	0x1B	0x1e
112	28	0x1C	0x3c

.ASM CODE OF TEST PROGRAM 2 :

```
# temp = A & B
# result = temp | C
# store result at 112
# Expected: M[112] = 11 (0x0B)

        .text
        .globl main

main:
    # initialize inputs
    addi $t0, $zero, 15
    sw   $t0, 100($zero)    # M[100] = A = 0x0F
    addi $t0, $zero, 3
    sw   $t0, 104($zero)    # M[104] = B = 0x03
    addi $t0, $zero, 8
    sw   $t0, 108($zero)    # M[108] = C = 0x08

    # load
    lw   $t1, 100($zero)    # t1 = A
    lw   $t2, 104($zero)    # t2 = B
    lw   $t3, 108($zero)    # t3 = C

    # nops for pipeline safety if required
    sll  $zero, $zero, 0
    sll  $zero, $zero, 0

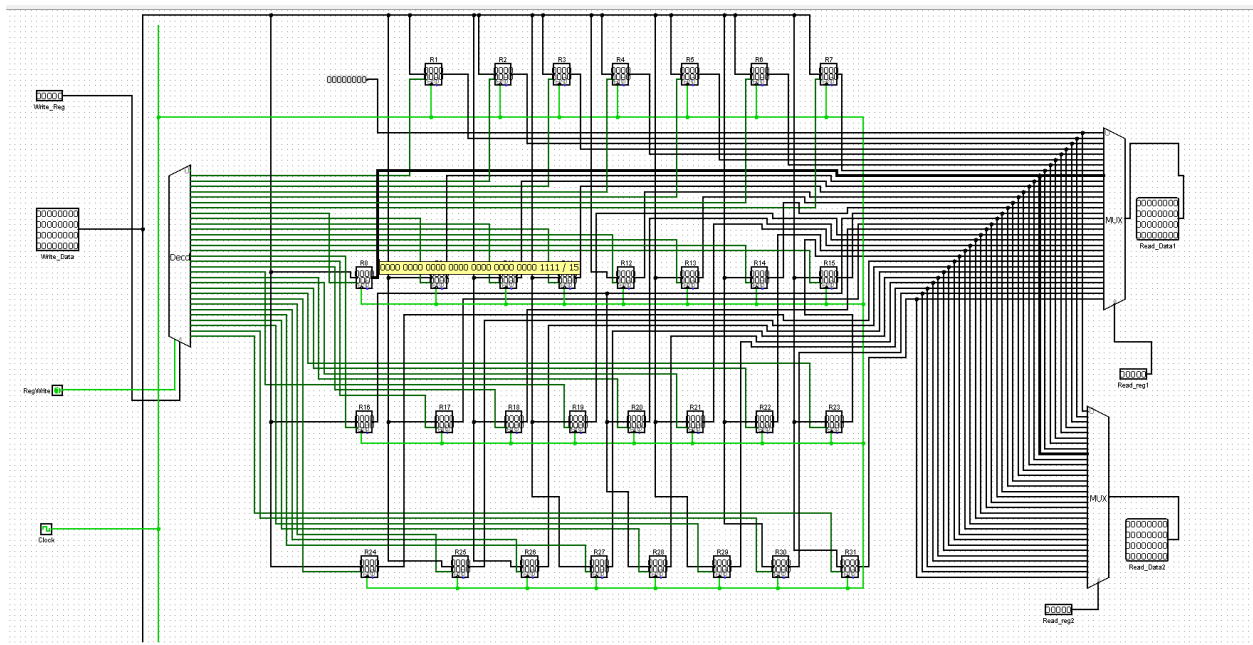
    # temp = A & B
    and  $t4, $t1, $t2      # t4 = 0x03

    # result = temp | C
    or   $t5, $t4, $t3      # t5 = 0x0B

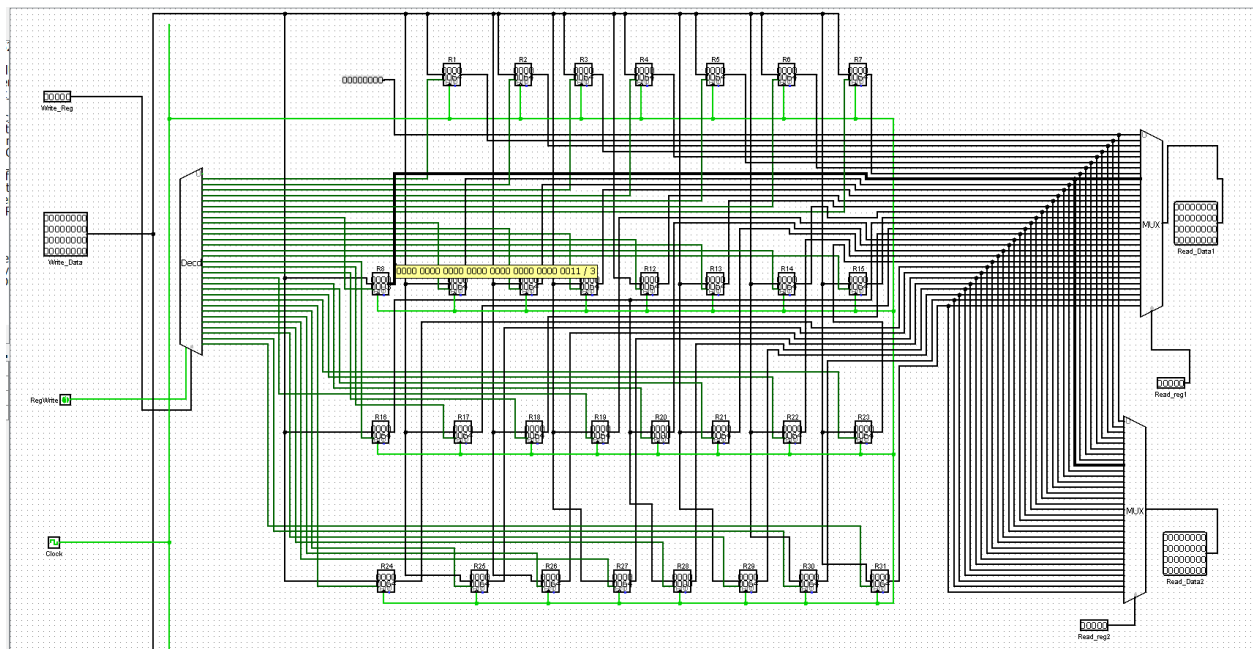
    # store result
    sw   $t5, 112($zero)    # M[112] = result

# infinite halt
halt:
    beq  $zero, $zero, halt
```

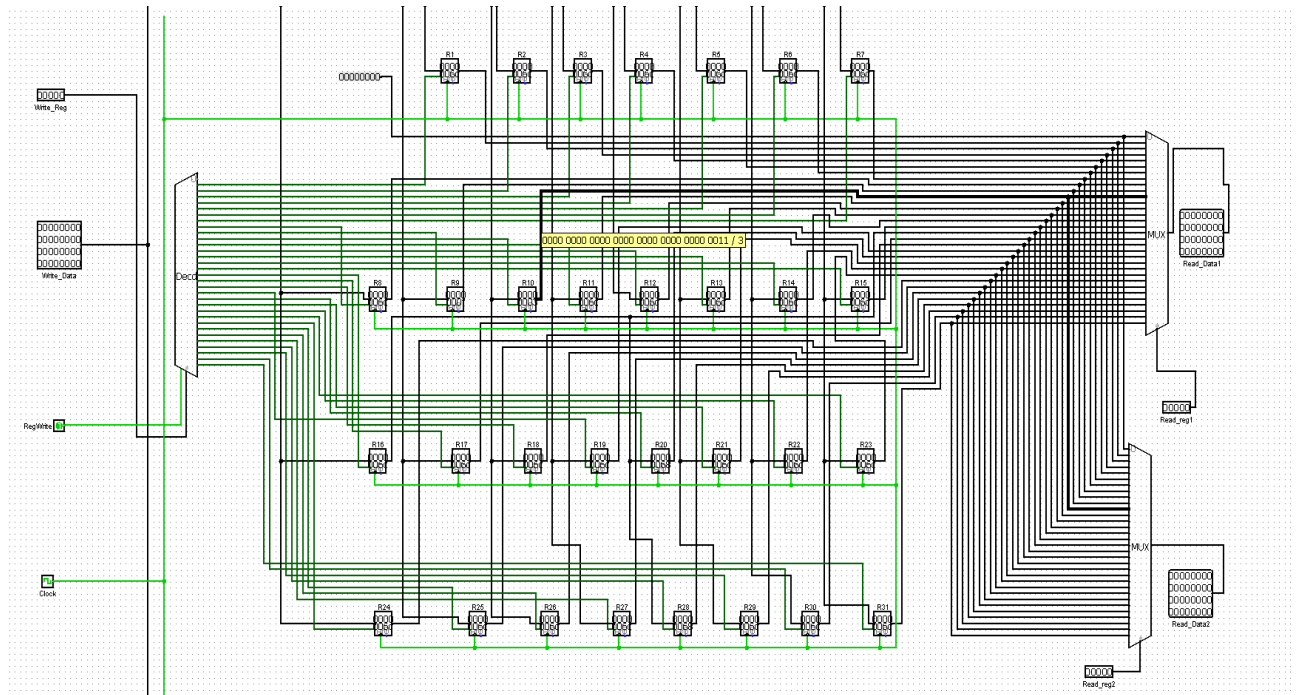
REGISTER FILE SIMULATION PICTURES :



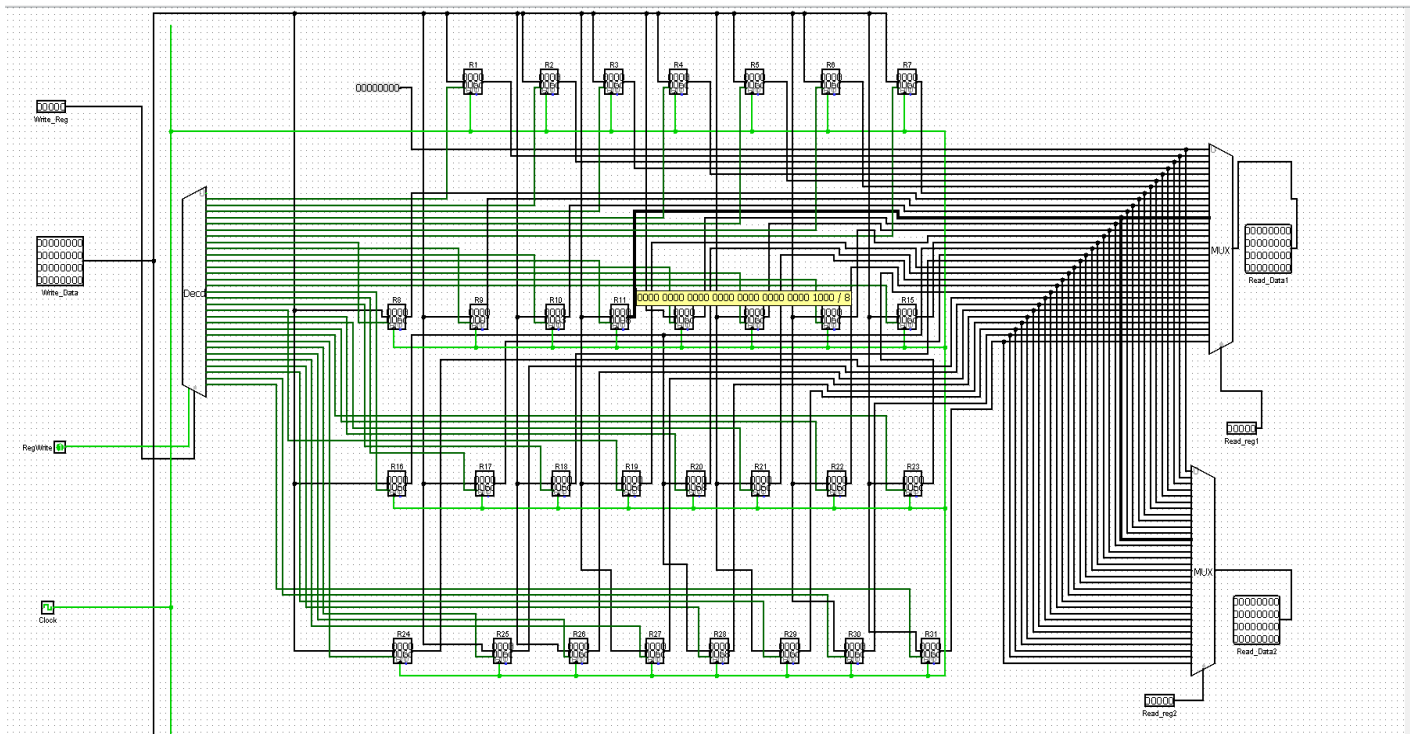
After executing `addi $t0, $zero, 15`
 \$t0 (R8) should contain 15



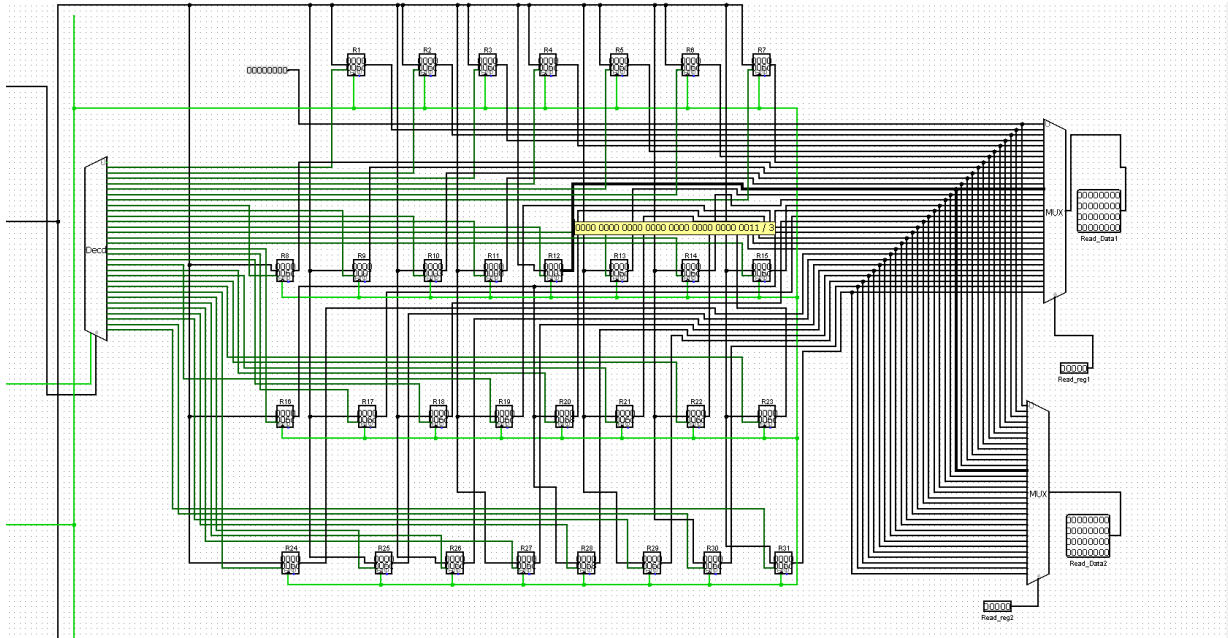
After executing `addi $t0, $zero, 3`
 \$t0(R8) should contain 3



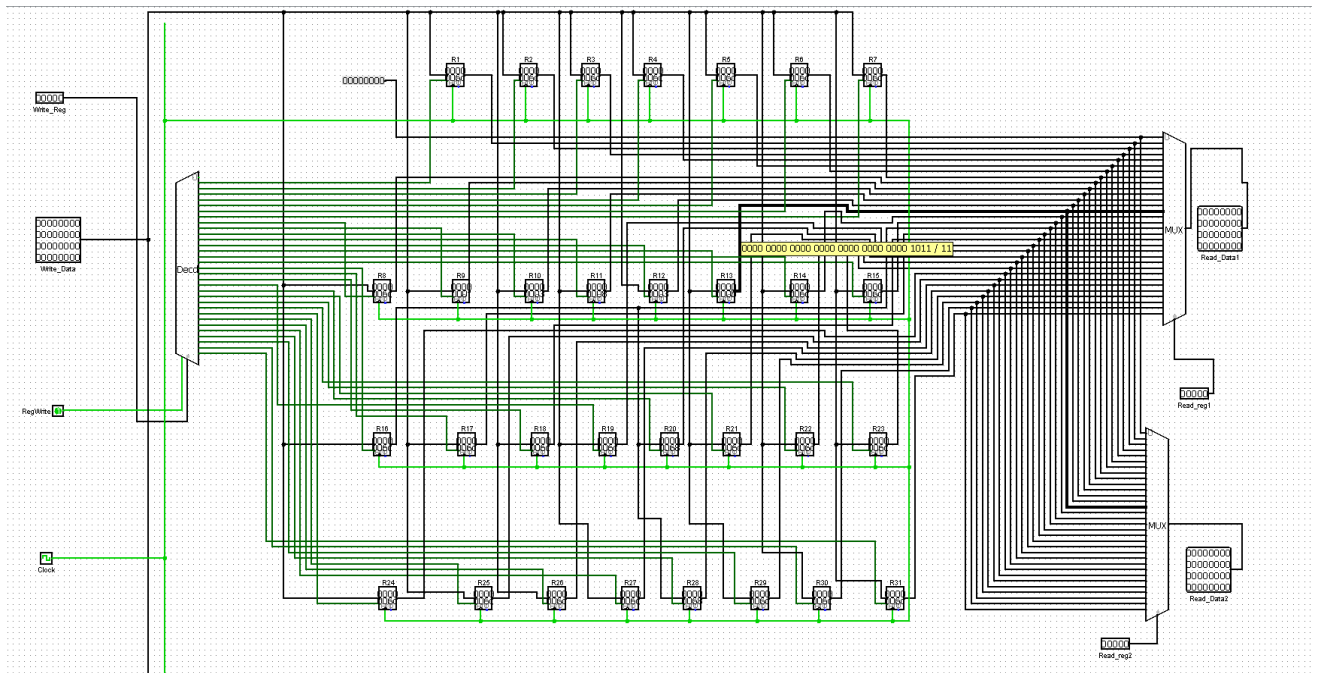
After Executing `lw $t2, 104($zero)`
`$t2 (R10)` should contain 3



After Executing `lw $t3, 108($zero)`
`$t3 (R11)` should contain 8.



After Executing and \$t4, \$t1, \$t2
 \$t4 (R12) should contain value 3 ($\$t4 = 15 \& 3 = 3$)



After Executing or \$t5, \$t4, \$t3
 \$t5 (R13) contains 11 ($\$t5 = 3 \mid 11 = 11$)

DATA MEMORY SIMULATION :

```

000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
008 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
010 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
018 00000000 0000000f 00000003 00000008 0000000b 00000000 00000000 00000000
020 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
028 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
030 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
038 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
040 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
048 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
050 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
058 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
060 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
068 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

```

Byte address (Decimal)	Word Address (Decimal)	Word Address (Hex)	Stored Value (Hex)
100	25	0x19	0x0F
104	26	0x1A	0x03
108	27	0x1B	0x08
112	28	0x1C	0x0B

.ASM CODE OF TEST PROGRAM 3

```
.text
.globl main

main:
    # Initialize test values
    addi $t0, $zero, 5    # A = 5
    addi $t1, $zero, 5    # B = 5
    addi $t2, $zero, 0    # clear t2

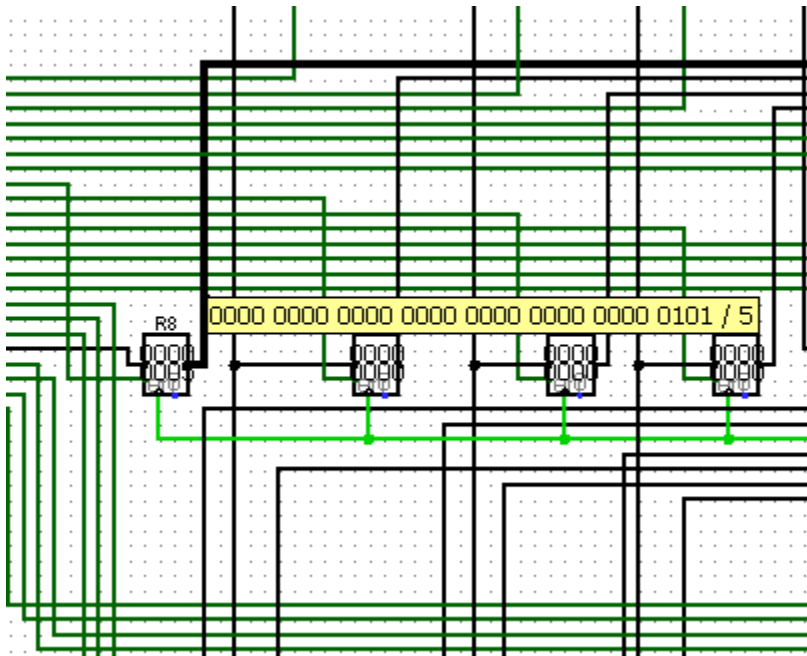
    # if (A == B) branch to skip the next addi
    beq $t0, $t1, equal_case

    # this runs only if not equal
    addi $t2, $zero, 9
    sw $t2, 200($zero)    # M[200] = 9 (not equal case)

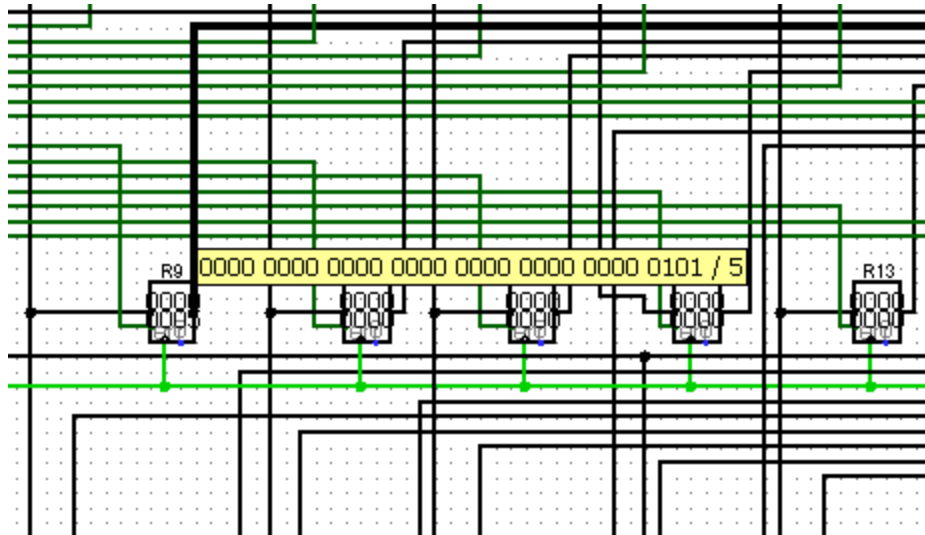
equal_case:
    addi $t2, $zero, 1
    sw $t2, 204($zero)    # M[204] = 1 (equal case)

halt:
    beq $zero, $zero, halt
```

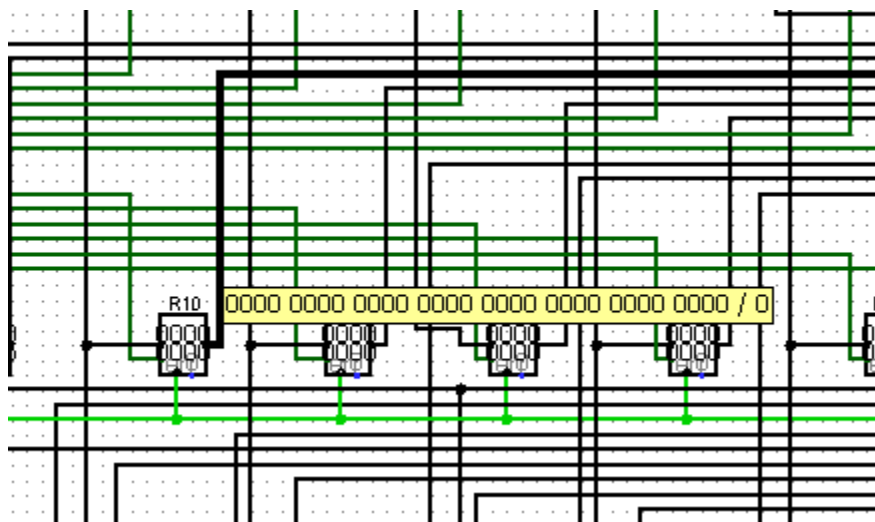
REGISTER FILE SIMULATION :



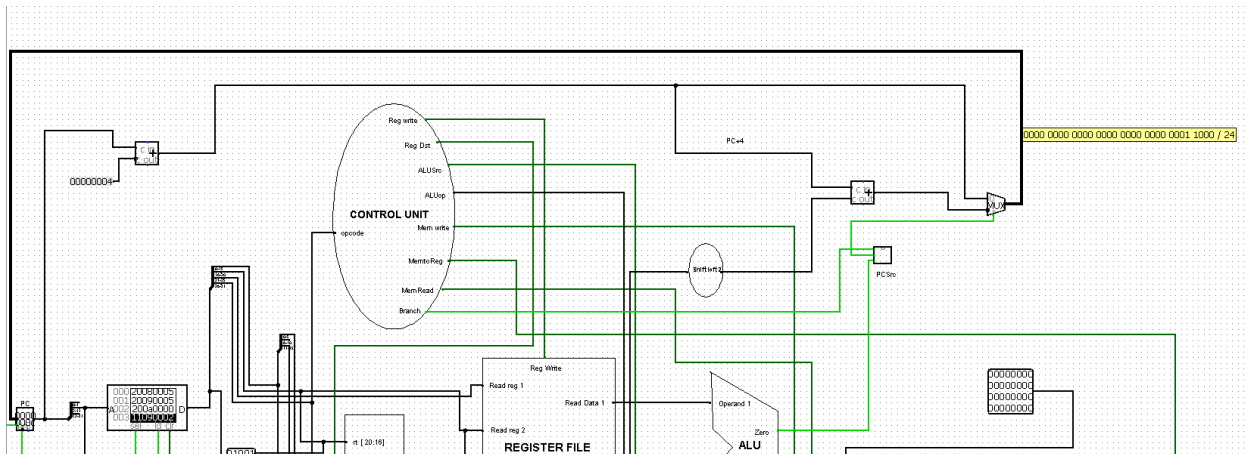
After Executing `addi $t0, $zero, 5`
\$t0 (R8) should contain value 5



After Executing `addi $t1, $zero, 5`
 \$t0 (R9) should contain value 5



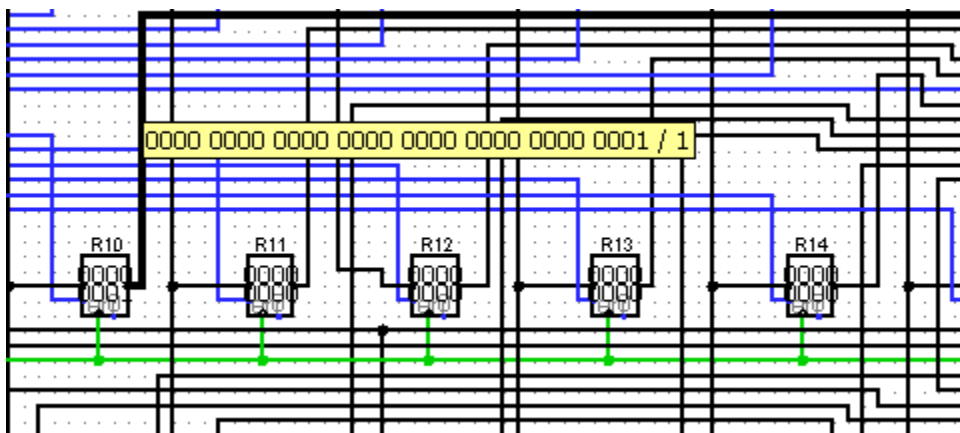
After Executing `addi $t2, $zero, 0`
 \$t0 (R10) should contain value 0



While executing beq \$t0,\$t1,equal_case (branch signal = 1 , zero = 1) so branching takes place.

So instead of 16 , PC has 24 in it due to branching

So the next instruction to execute is addi \$t2,\$zero,1



After Executing addi \$t2,\$zero,1
\$t2 has value 10 in it.

DATA MEMORY SIMULATION :

000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
008	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
010	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
018	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
020	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
028	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
030	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
038	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
040	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
048	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
050	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
058	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
060	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000

Thus, the word 0x00000001 from \$t2 is written to memory locations 204-207 (word address in decimal 51 or word address in hex 0x33).

TAKE AWAY'S FROM ASSIGNMENT :

This assignment helped me understand how different components of a MIPS processor such as the Program Counter, Instruction Memory, Data Memory, and Control Unit work together to complete the instruction execution cycle.

I learned how instructions are fetched, decoded, executed, and how data moves between registers and memory.

Implementing memory access and PC update logic clarified how branching and sequential execution are managed in hardware.

Overall, this task strengthened my understanding of processor datapath design and the importance of precise control signal coordination.

REFERENCES :

1. MIPS diagram attached in Lecture slides
2. Patterson, D. A., & Hennessy, J. L. (2014). Computer Organization and Design
3. Logisim Reference Guide